

# **Chapter 1**

## **Introduction to Object Oriented Programming**

**[using Java]**

# Programming Paradigms :

## Programming languages

---

- ▶ A Programming language is a notational system for describing tasks/computations in a machine and human readable form.
- ▶ Most computer languages are designed to facilitate certain operations and not others: **numerical computation, or text manipulation, or I/O.**
- ▶ More broadly, a computer language typically embodies a particular *programming paradigm*.

# Programming Paradigms :

## Programming languages

---

- ▶ Which programming language?
- ▶ Since a task can be solved in different ways (paradigms), the language used to describe the solution differs in abstractions, structures due to the way in which the problem is solved.
- A **programming paradigm** is a fundamental style of computer programming.
- Programming paradigms differ in:
  - the **concepts** and **abstractions** used to represent the elements of a program (such as objects, functions, variables, constraints, etc.)
  - the steps that **compose a computation** (assignment, evaluation, data flow, control flow, etc.).

# Programming Paradigms : Programming languages

---

## ► Classification

| Imperative/<br>Algorithmic                     | Declarative                             |                      | Object<br>Oriented                           |
|------------------------------------------------|-----------------------------------------|----------------------|----------------------------------------------|
|                                                | Functional<br>Programming               | Logic<br>Programming |                                              |
| Algol<br>Cobol<br>PL/I<br>Ada<br>C<br>Modula-3 | Lisp<br>Haskell<br>ML<br>Miranda<br>APL | Prolog               | Smalltalk<br>Simula<br>C++ , C#<br>Java , VB |

# Imperative paradigms

---

- ▶ It is **based on commands** that update variables in storage. The Latin word *imperare* means “to command”. E.g *GO TO* command
- ▶ The language provides statements, such as *assignment statements, which explicitly change the state of the memory of the computer.*
- ▶ This model closely matches the actual executions of computer and usually has high execution efficiency.
- ▶ Many people also find the imperative paradigm to be a more natural way of expressing themselves.

# Imperative paradigms ...

---

- ▶ *Advantages :*

- ▶ efficient;
- ▶ close to the machine;
- ▶ popular;
- ▶ familiar.

- ▶ *Disadvantages :*

- ▶ The semantics of a program can be complex to understand or prove
- ▶ debugging is harder;
- ▶ Abstraction is more limited than with some paradigms;
- ▶ Order is crucial, which doesn't always suit itself to problems.

# Functional programming paradigms

---

- ▶ In this paradigm we express computations as the evaluation of mathematical functions.
- ▶ Functional programming paradigms treat values as single entities. Unlike variables, values are never modified. Instead, values are transformed into new values.
- ▶ Computations of functional languages are performed largely through applying functions to values,.

# Functional programming paradigms...

---

## Advantages:

- The high level of abstraction, removes the possibility of committing many classes of errors;
- The lack of dependence on assignment operations, allowing programs to be evaluated in many different orders. This evaluation order independence makes function-oriented languages good candidates for programming massively parallel computers;
- The absence of assignment operations makes the function-oriented programs much more amenable to mathematical proof and analysis than are imperative programs.

## Disadvantages:

- ▶ Perhaps less efficiency
- ▶ Problems involving many variables or a lot of sequential activity are sometimes easier to handle imperatively or with object-oriented programming.



# Logic programming paradigms

---

- ▶ In this paradigm we express computation in exclusively in terms of ***mathematical logic***.
- ▶ While the functional paradigm emphasizes the idea of a mathematical function, the **logic paradigm focuses on predicate logic**, in which the basic concept is a *relation*.
- ▶ Logic languages are useful for expressing problems where it is not obvious what the functions should be.

# Logic programming paradigms...

---

- A logical program is divided into three sections:
  - ▶ a series of definitions/declarations that define the problem domain
  - ▶ statements of relevant facts
  - ▶ statement of goals in the form of a query
- Any deducible solution to a query is returned. The definitions and declarations are constructed entirely from relations. i.e.  $X$  is a member of  $Y$  or  $X$  is in the interval between  $a$  and  $b$  etc.

## ***Advantages:***

- system solves the problem, so the programming steps themselves are kept to a minimum;
- Proving the validity of a given program is simple.

# The Object-Oriented Paradigm

---

- ▶ *Object Oriented Programming (OOP)* is a paradigm in which real-world objects are each viewed as separate entities having their own state which is modified only by built in procedures, called methods.

**Alan Kay** characterized the fundamental of OOP as follows:

- ▶ Everything is modeled as object
- ▶ Computation is performed by message passing: objects communicate with one another via message passing.
- ▶ Every object is an instance of a class where a class represents a grouping of similar objects.

# The Object-Oriented Paradigm

---

- Inheritance: defines the relationships between classes.
- Inheritance gives OOP its chief benefit over other programming paradigms - relatively easy code reuse and extension without the need to change existing source code.
- The Object Oriented paradigm focuses on the objects that a program is representing, and on allowing them to exhibit "behavior".
- Unlike imperative paradigm, where data are passive and procedures are active, in the O-O paradigm data is combined with procedures to give objects, which are thereby rendered active.

# Fundamental Principles of OOP

---

1. Objects
2. Classes
3. Methods and Messages
4. Inheritance
5. Encapsulation
6. Polymorphism

# OOP Concepts

---

- ▶ In object-oriented programming (OOP), programs are organized into **objects**
- ▶ The properties of objects are determined by their **class**
- ▶ Objects act on each other by passing **messages**

# Object

---

- ▶ Definition: An object is a software bundle that has ***State*** and ***Behavior***.
- ▶ Software Objects are often used to model real-world objects.
- ▶ Example: dogs have states (name, color, hungry, breed) and behaviors (bark, fetch, and wag tail).

# Object Examples

---

- ▶ **Example 1: Dogs**

- ▶ States: name, color, breed, and “is hungry?”
- ▶ Behaviors: bark, run, and wag tail

- ▶ **Example 2: Cars**

- ▶ States: color, model, speed, direction
- ▶ Behaviors: accelerate, turn, change gears



# Class

---

- ▶ Definition: A class is a blueprint that defines the states and the behaviors common to all objects of a certain kind.
- ▶ In the real world, you often have many objects of the same kind. For example, a guard dog, herding dog, snoop dog ...
- ▶ Even though all dogs have four legs, and bark, each dog's behavior is independent of other dogs.
- ▶ For example: Dog #1 is a black Poodle, Dog #2 is a red Irish Setter

# Methods

---

- ▶ An Object's behavior is defined by its **methods**
- ▶ Methods, like fields, are written inside the braces of the class
- ▶ Methods can access the fields (the state) of their object and can change them

# Message

---

- ▶ Definition: Software objects interact and communicate with each other by **sending *messages*** to each other.

# Inheritance

---

- ▶ Inheritance allows child classes inherits the characteristics of existing parent class
  - ▶ Attributes (fields and properties)
  - ▶ Operations (methods)
- ▶ Child class can extend the parent class
  - ▶ Add new fields and methods
  - ▶ Redefine methods (modify existing behavior)
- ▶ A class can implement an interface by providing implementation for all its methods

# Types of Inheritance

---

## ► Inheritance terminology

**derived class**

**inherits**

**base class /  
parent class**

**class**

**implements**

**interface**

**derived  
interface**

**implements**

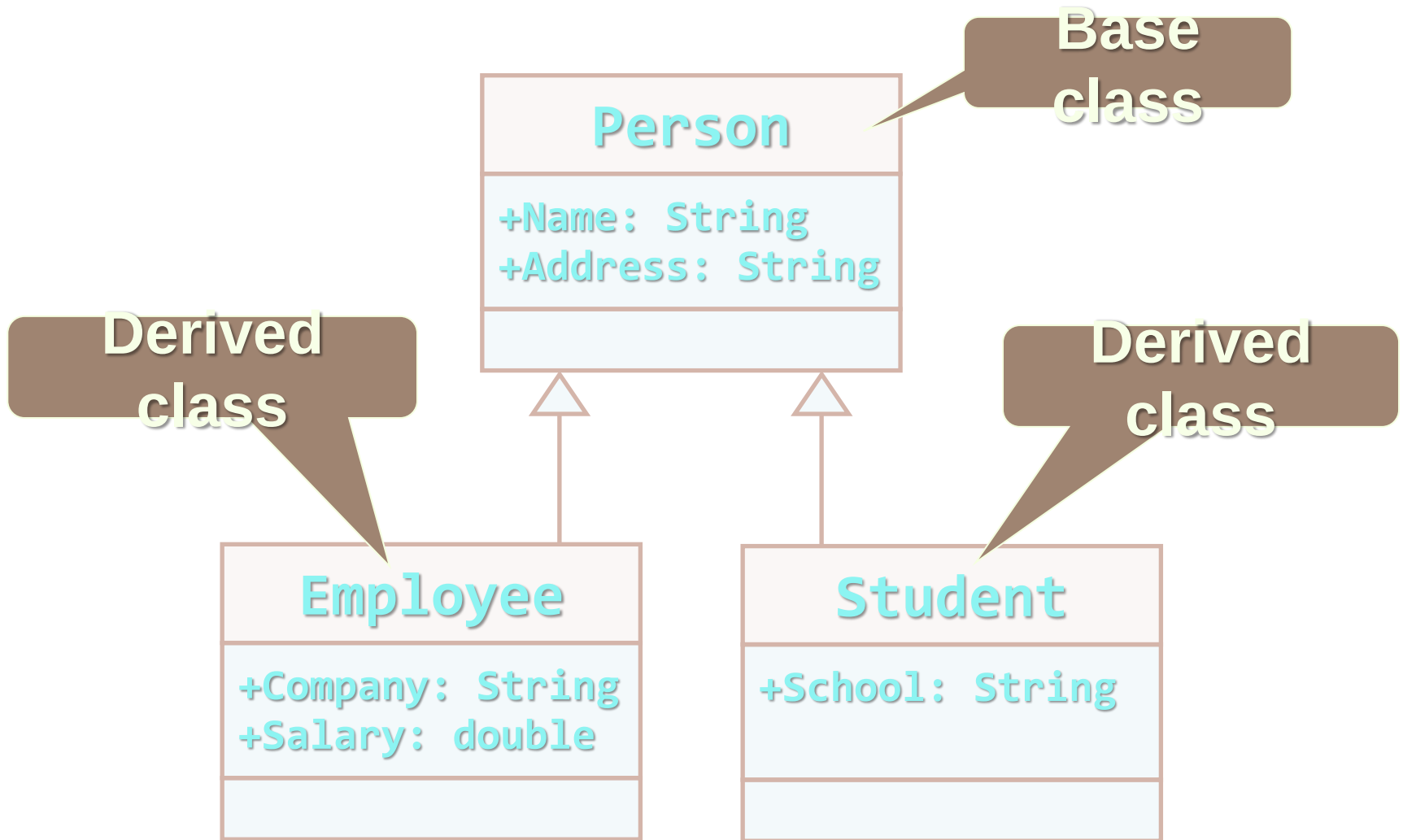
**base interface**

# Inheritance – Benefits

---

- ▶ Inheritance has a lot of benefits
  - ▶ Extensibility
  - ▶ Reusability
  - ▶ Provides abstraction
  - ▶ Eliminates redundant code
- ▶ Use inheritance for building is-a relationships
  - ▶ E.g. dog is-a animal (dogs are kind of animals)
- ▶ Don't use it to build has-a relationship
  - ▶ E.g. dog has-a name (dog is not kind of name)

# Inheritance – Example



# Encapsulation

---

- ▶ Encapsulation hides the implementation details
- ▶ Class announces some operations (methods) available for its clients – its public interface
- ▶ All data members (fields) of a class should be hidden
  - ▶ Accessed via properties (read-only and read-write)
- ▶ No interface members should be hidden



# Encapsulation – Example

---

- ▶ Data fields are private
- ▶ Constructors and accessors are defined (getters and setters)

| Person                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------|
| <b>-name : string</b><br><b>-age : TimeSpan</b>                                                                     |
| <b>+Person(string name, int age)</b><br><b>+Name : string { get; set; }</b><br><b>+Age : TimeSpan { get; set; }</b> |

# Encapsulation – Benefits

---

- ▶ Ensures that structural changes remain local:
  - ▶ Changing the class internals does not affect any code outside of the class
  - ▶ Changing methods' implementation does not reflect the clients using them
- ▶ Encapsulation allows adding some logic when accessing client's data
  - ▶ E.g. validation on modifying a property value
- ▶ Hiding implementation details reduces complexity → easier maintenance

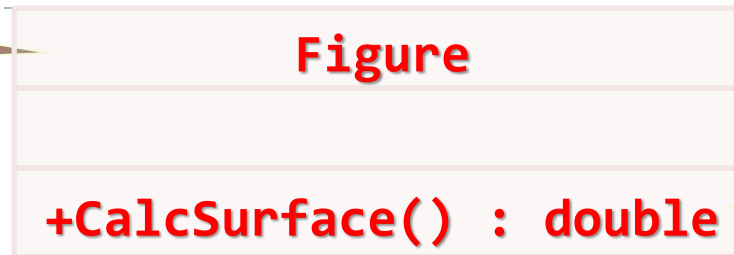
# Polymorphism

---

- ▶ Polymorphism = ability to take more than one form (objects have more than one type)
  - ▶ A child class may override some of the behaviors of the parent class
- ▶ Polymorphism allows abstract operations to be defined and used
  - ▶ Abstract operations are defined in the base class' interface and implemented in the child classes
    - ▶ Declared as abstract or virtual

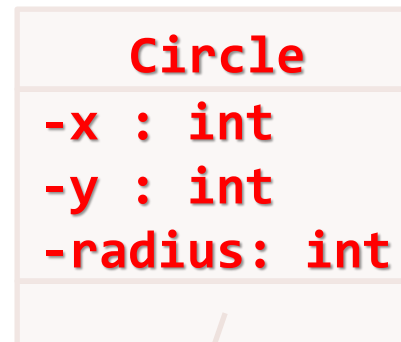
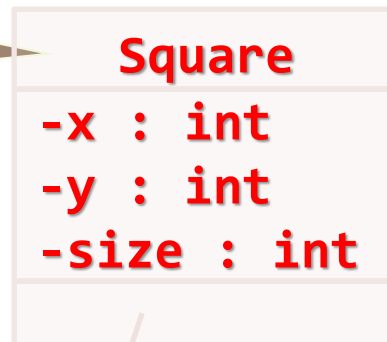
# Polymorphism – Example

Abstract  
class



Abstract  
action

Concrete  
class



Override  
n action

```
override CalcSurface()  
{  
    return size * size;  
}
```

Override  
n action

```
override CalcSurface()  
{  
    return PI * radius * radius;  
}
```

# Overview of Java Programming

---

- ▶ Java is an **object-oriented** programming language
- ▶ The Java programming language was created by Sun Microsystems, Inc.
- ▶ It was introduced in 1995 and its popularity has grown quickly since
- ▶ Its primary feature is that it could function nicely in a networked environment.
- ▶ The explosion of WWW created an environment in which the language could live.

# Overview of Java Programming

---

- ▶ Java technology can be seen as both a **language** and a **platform**.
- ▶ Using it you can write applications that can run on practically any device, including a PC, PDA, a cellular phone or a television.
- ▶ The Java platform is formed from two components:
  - ▶ **The Java application programming interface (Java API)**
    - ▶ Set of libraries that are used to accomplish tasks such as creating GUIs, performing file I/O and establishing network comm.
  - ▶ **The Java Virtual Machine (JVM)**
    - ▶ Is in charge of executing your code in a specific environment.

# Characteristics of Java

---

- ▶ Java Is Simple
- ▶ Java Is Object-Oriented
- ▶ Java Is Distributed
- ▶ Java Is Interpreted
- ▶ Java Is Robust
- ▶ Java Is Secure
- ▶ Java Is Architecture-Neutral
- ▶ Java Is Portable
- ▶ Java's Performance
- ▶ Java Is Multithreaded
- ▶ Java Is Dynamic

# Java Program Structure

---

- ▶ In the Java programming language:
  - ▶ A program is made up of one or more *classes*
  - ▶ A class contains one or more *methods*
  - ▶ A method contains program *statements*
- ▶ These terms will be explored in detail throughout the course
- ▶ A Java application always contains a method called `main`



# Java Program Structure ...

---

```
// comments about the class
```

```
public class MyProgram
```

```
{
```

```
    // comments about the method
```

```
    public static void main (String[] args)
```

```
    {
```

} method body

```
    }
```

method header

```
}
```

# Development Environments

---

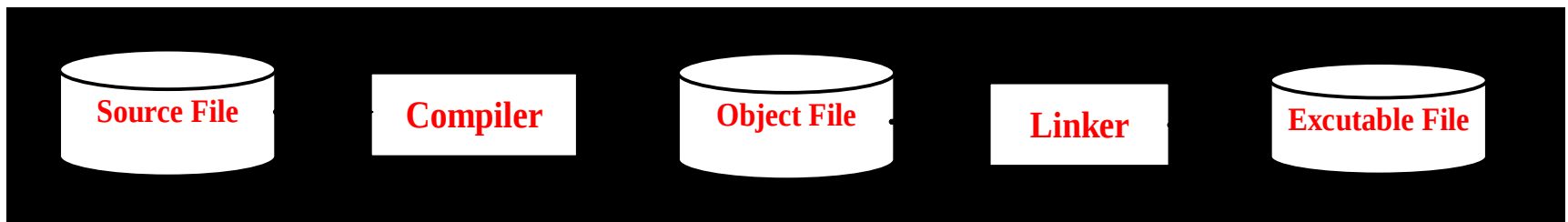
- ▶ There are many programs that support the development of Java software, including:
  - ▶ Sun Java Development Kit (JDK)
  - ▶ Sun NetBeans
  - ▶ IBM Eclipse
  - ▶ Borland JBuilder
  - ▶ MetroWerks CodeWarrior
  - ▶ BlueJ
  - ▶ jGRASP

# Compiling Source Code

A program written in a high-level language like java is called a *source program*.

Since a computer cannot understand a source program. Program called a **compiler** is used to translate the source program into a machine language program called an *object program*.

The object program is often then linked with other supporting library code before the object can be executed on the machine.

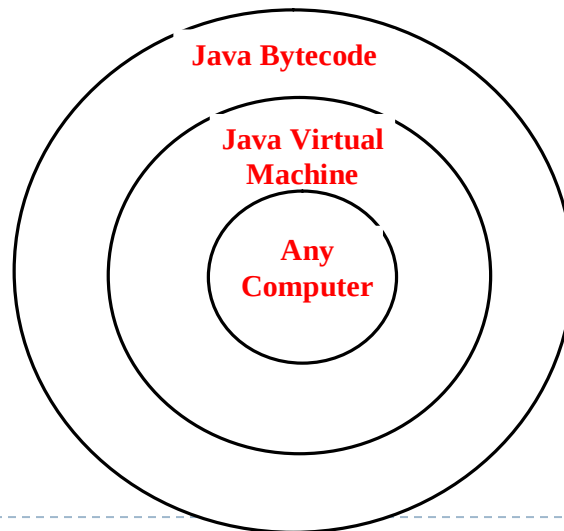


# Compiling Java Source Code

With Java, you write the program once, and compile the source program into a special type of object code, known as *bytecode*.

The bytecode can then run on any computer with a Java Virtual Machine, as shown below.

Java Virtual Machine is a software that interprets Java bytecode.



# Running Programs

---

- ▶ All programs follow a simple format:

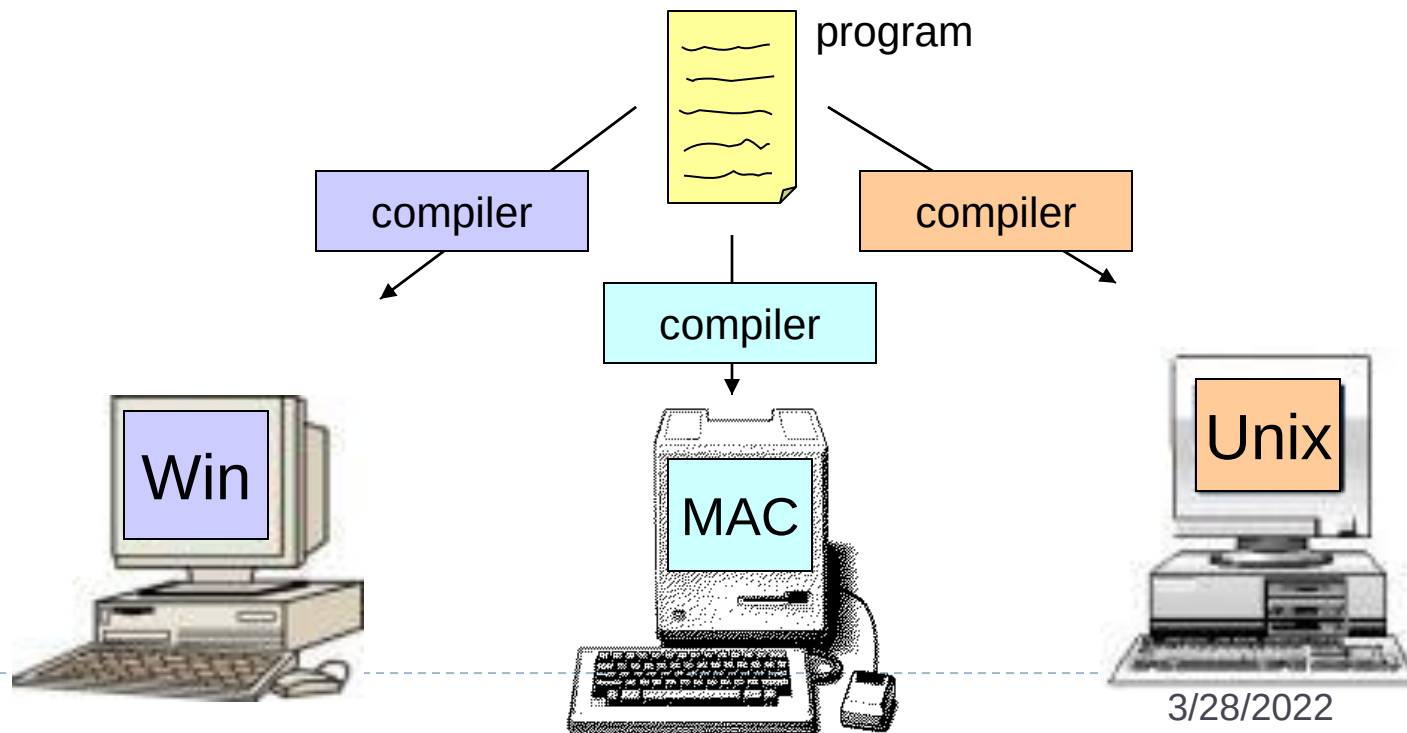
Input      →      Execution      →      Output

- ▶ Inputs can be from users, files, or other computer programs
- ▶ Outputs can take on many forms: numbers, text, graphics, sound, or commands to other programs

# Multiple Compilers

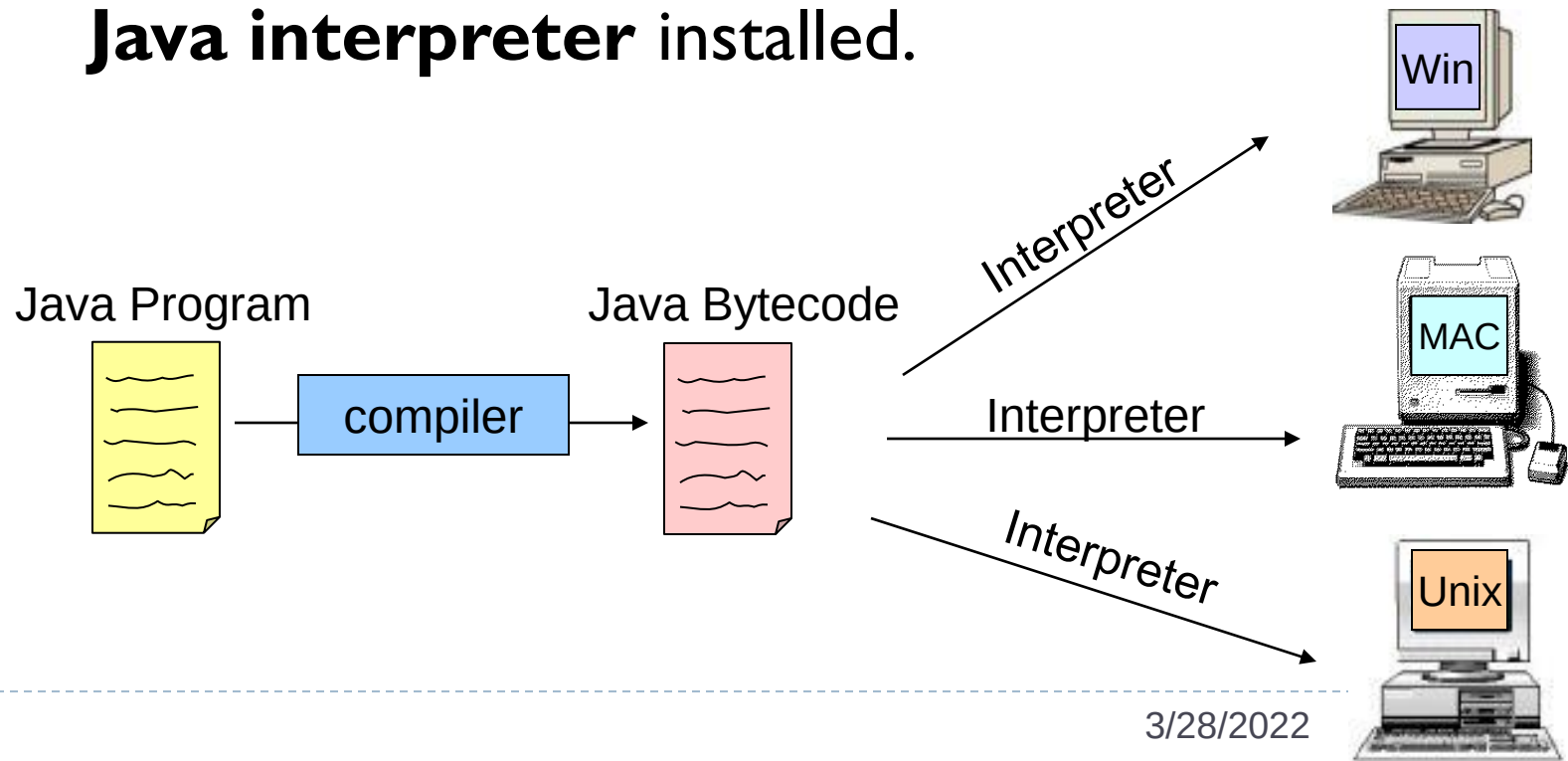
---

- ▶ Because different operating systems (Windows, Macs, Unix) require different machine code, you must compile most programming languages separately for each platform.



# Java Interpreter

- ▶ **Java** is a little different.
- ▶ Java compiler produces **bytecode** not machine code.
- ▶ Bytecode can be run on any computer with the **Java interpreter** installed.



# Your First Java Program

---

- ▶ Open your text-editor and type the following piece of Java code exactly:

```
class HelloWorld {  
    public static void main(String[] args)  
    {  
        System.out.println("Hello World!");  
    }  
}
```

- ▶ Save this file as HelloWorld.java (watch capitalization) in the following directory:  
c:\java



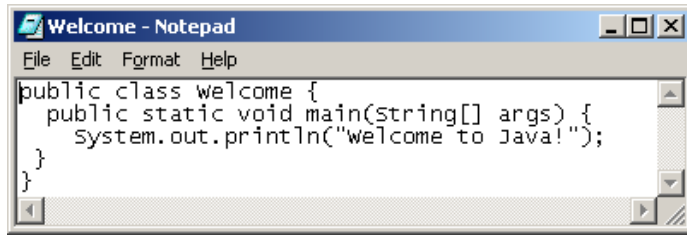
# Compiling and Running Your First Program

---

- ▶ Open the command prompt in Windows
- ▶ To run the program that you just wrote, type at the command prompt:  
`cd c:\java`
- ▶ Your command prompt should now look like this:  
`c:\java>`
- ▶ To compile the program that you wrote, you need to run the **Java Development Tool Kit Compiler** as follows:  
At the command prompt type:  
`c:\java> javac HelloWorld.java`
- ▶ You have now created your first compiled Java program named HelloWorld.class
- ▶ To run your first program, type the following at the command prompt:  
`c:\java> java HelloWorld`

*Although the file name includes the .class extension , this part of the name must be left off when running the program with the Java interpreter.*

# Creating, Compiling, and Running Programs



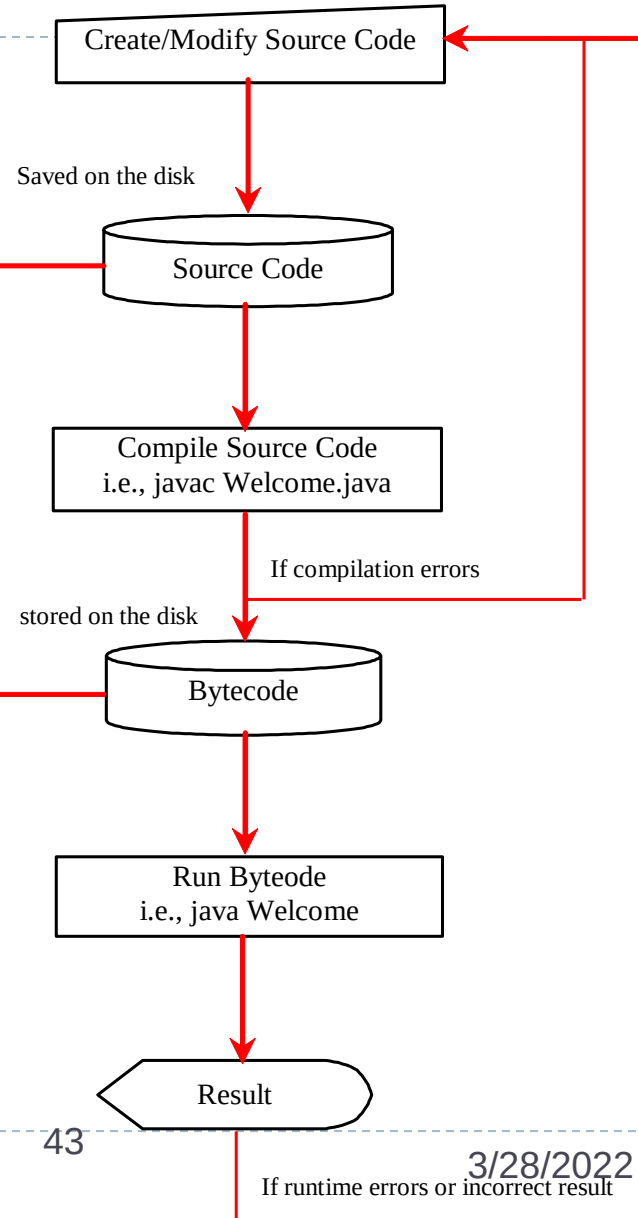
```
public class welcome {  
    public static void main(String[] args) {  
        System.out.println("welcome to Java!");  
    }  
}
```

Source code (developed by the programmer)

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Byte code (generated by the compiler for JVM to read and interpret, not for you to understand)

```
...  
Method Welcome()  
  0 aload_0  
  ...  
Method void main(java.lang.String[])  
  0 getstatic #2 ...  
  3 ldc #3 <String "Welcome to Java!">  
  5 invokevirtual #4 ...  
  8 return
```



# Types of Java Program

---

- ▶ All Java programs can be classified as Applications and Applets.
- ▶ Java applications are large programs that run stand-alone, with the support of a virtual machine.
- ▶ Applet is a relatively small Java program executed under the by a browser (inside an appletviewer).
- ▶ **Applets are great for creating dynamic and interactive web applications**

# Types of Java Program...

---

- ▶ Applets on client-side and servlets on server-side makes Java a truly "**Internet-based language**".
- ▶ To execute applets, the browsers come with JRE (Java Runtime Environment). The browsers with Java Runtime Environment (or to say, JVM) loaded are known as **Java enabled browsers**.
- ▶ Note: Browser do not have a Java compiler as a compiled applet file (.class file) is given to browser to execute.

# Types of Java Program...

---

## **Advantages of Applets :**

- ▶ Execution of applets is easy in a Web browser and does not require any installation or deployment procedure in realtime programming (where as servlets require).
- ▶ Writing and displaying (just opening in a browser) graphics and animations is easier than applications.
- ▶ In GUI development, constructor, size of frame, window closing code etc. are not required (but are required in applications).

# Types of Java Program...

| Feature       | Application                                                                          | Applet                                                          |
|---------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| main() method | Present                                                                              | Not present                                                     |
| Execution     | Requires JRE                                                                         | Requires a browser like Chrome                                  |
| Nature        | Called as stand-alone application as application can be executed from command prompt | Requires some third party tool help like a browser to execute   |
| Restrictions  | Can access any data or software available on the system                              | cannot access any thing on the system except browser's services |
| Security      | Does not require any security                                                        | Requires highest security for the system as they are untrusted  |